UNIVERSITY PAYROLL MANAGEMENT SYSTEM

PROJECT REPORT

Course: Python Essentials

Project Type: Command-Line Python Application

Student Name: SATYAM

Registration Number: 26MIM10054

Academic Year: 2026

1. Abstract

The University Payroll Management System is a beginner-level command-line application developed in Python to manage basic university employee payroll operations. The system supports faculty and staff records and provides functions for adding employees, viewing records, searching employees, recording overtime, calculating payroll, generating payslips, creating payroll reports, and removing employees.

The project is designed to demonstrate core Python Essentials concepts such as variables, lists, dictionaries, functions, loops, conditional statements, exception handling, input/output, and arithmetic calculations. It can be executed entirely from a terminal without a graphical interface or third-party packages.

2. Introduction

Payroll is an important administrative activity in a university because employee salaries must be calculated consistently and deductions must be applied correctly. A manual process can become difficult to manage as the number of employees increases.

This project provides a small educational payroll solution. It is intentionally simple so that a beginner can understand the complete program and explain its implementation during project evaluation or viva.

3. Problem Statement

Universities need a straightforward way to maintain employee salary information and perform monthly payroll calculations. Manual calculations can result in mistakes in overtime, tax, deductions, and final salary. The proposed system automates these basic calculations through a menu-driven Python program.

4. Objectives

- Manage university faculty and staff employee records.
- Assign a unique employee ID during the current program session.
- Calculate monthly basic salary and overtime pay.
- Apply different tax rates for faculty and staff.
- Calculate retirement deductions and net salary.
- Generate individual employee payslips.
- Generate a summary payroll report.
- Search and remove employee records.
- Validate user input and handle common input errors.
- Provide a completely terminal-based application.

5. Scope of the Project

The system focuses on basic payroll management. It includes employee details, salary, employee type, overtime, deductions, payslips, searching, removal, and summary reporting. The current implementation stores records in memory using a Python list, so the data exists only while the application is running.

The project does not attempt to implement a complete enterprise university payroll platform. Authentication, permanent database storage, attendance, leave management, and integration with banking or ERP systems are outside the current scope.

6. Functional Requirements

Requirement	Description
FR1 - Add Employee	Add name, department, role, employee type, and salary.
FR2 - View Employees	Display all employees stored during the current session.
FR3 - Search Employee	Search using employee name or department.

FR4 - Add Overtime	Record additional overtime hours for an employee.
FR5 - Calculate Payroll	Calculate gross salary, tax, retirement, and net salary.
FR6 - Generate Payslip	Display detailed salary information for an employee.
FR7 - Payroll Report	Display payroll calculations and totals for all employees.
FR8 - Remove Employee	Delete an employee using the employee ID.

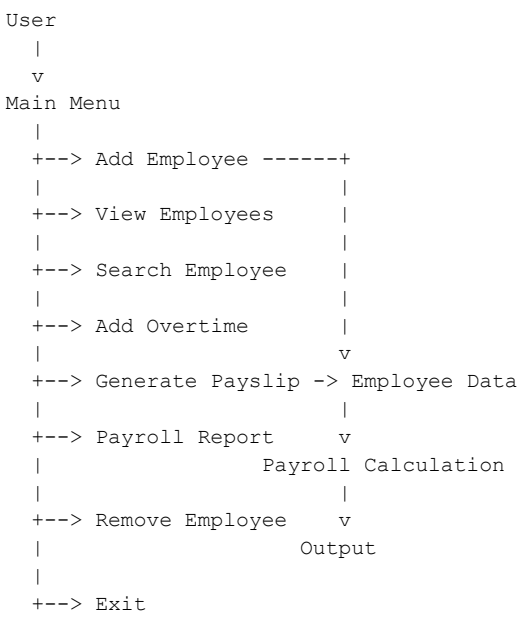
7. Non-Functional Requirements

- Usability: The menu-driven interface should be understandable to a beginner.
- Reliability: Invalid numeric input and missing employee IDs are handled without unexpected program termination.
- Maintainability: Major operations are separated into functions with clear responsibilities.
- Portability: The application runs from a terminal on systems with Python installed.
- Performance: The in-memory approach provides quick responses for a small educational dataset.
- Dependency simplicity: The application uses Python standard-library functionality only.

8. System Design

The application follows a simple procedural design. A main menu controls the flow and calls separate functions for each major operation. Employee records are represented by dictionaries and stored in an employees list.

High-level flow:



9. Program Modules / Functions

Function	Purpose
add_employee()	Collects and validates employee information and adds a dictionary to the list.
view_employees()	Displays all current employee records.
find_employee()	Returns an employee matching a given ID.
add_overtime()	Adds overtime hours after validating the employee ID and hours.
calculate_payroll()	Performs salary, overtime, tax, retirement, and net salary calculations.
generate_payslip()	Displays detailed payroll information for one employee.
search_employee()	Searches employee name or department.
remove_employee()	Removes an employee record.
payroll_report()	Calculates and displays payroll totals.
main()	Runs the menu repeatedly until the user exits.

10. Data Structure

The system uses a list named employees. Each employee is represented by a dictionary containing the following fields:

Field	Meaning
id	Employee identifier
name	Employee name
department	University department
role	Job position
type	Faculty or Staff
salary	Monthly basic salary
overtime	Recorded overtime hours

11. Payroll Calculation

- Hourly Rate = Monthly Salary / 160
- Overtime Pay = Overtime Hours × Hourly Rate × 1.5
- Gross Salary = Basic Salary + Overtime Pay
- Faculty Tax = Gross Salary × 10%
- Staff Tax = Gross Salary × 8%
- Retirement = Gross Salary × 5%
- Net Salary = Gross Salary - Tax - Retirement

12. Input Validation and Error Handling

The program checks for empty text fields, invalid employee type choices, invalid salary input, negative salary values, invalid employee IDs, invalid overtime input, negative overtime values, and missing employee records. Numeric conversions use try/except so that non-numeric input can be reported to the user instead of causing an unhandled exception.

13. User Interface

The application uses a terminal menu with eight options. This design satisfies the command-line execution requirement and avoids dependence on a graphical setup.

```
1. Add Employee
2. View Employees
3. Search Employee
4. Add Overtime
5. Generate Payslip
6. Payroll Report
7. Remove Employee
8. Exit
```

14. Sample Execution

```
==========================================
 UNIVERSITY PAYROLL MANAGEMENT SYSTEM
==========================================
1. Add Employee
2. View Employees
3. Search Employee
4. Add Overtime
5. Generate Payslip
6. Payroll Report
```

```
7. Remove Employee
8. Exit

Enter your choice: 1

Enter employee name: Ananya Sharma
Enter department: Computer Science
Enter job role: Professor

Employee Type:
1. Faculty
2. Staff

Enter choice: 1
Enter monthly salary: 85000

Employee added successfully!
```

15. Testing Plan

Test Case	Expected Result
Add valid Faculty employee	Employee is added and receives an ID.
Add valid Staff employee	Employee is added successfully.
Enter empty name	Program rejects the input.
Enter invalid salary	Program displays an error.
Enter negative salary	Program rejects the salary.
Add overtime to valid ID	Overtime hours are added.
Use invalid employee ID	Program reports employee not found.
Generate payslip	Salary and deductions are displayed.
Search by department	Matching employees are displayed.
Remove valid employee	Employee is removed.
Generate payroll report	Totals are calculated and displayed.

16. Installation and Execution

Requirements: Python 3.8 or later. No third-party packages are required.

Place the file in a project folder and open a terminal in that folder.

```
python --version
python university_payroll.py

# On systems using python3:
python3 university_payroll.py
```

17. Limitations

- Employee data is not permanently stored.
- There is no database connection.
- There is no authentication or authorization.
- Tax and retirement rates are fixed in the source code.
- There is no graphical or web interface.
- The application is intended for a small educational dataset.

18. Future Enhancements

- Add SQLite or another persistent database.
- Add administrator login and role-based access.
- Add attendance and leave management.
- Store monthly payroll history.

- Export reports to CSV and PDF.
- Make tax and deduction rules configurable.
- Add automated unit and integration tests.
- Develop an optional web or graphical interface.

19. Conclusion

The University Payroll Management System provides a practical example of applying Python Essentials concepts to a real-world administrative problem. The program demonstrates structured functions, data structures, validation, calculations, and command-line interaction. Its simple design makes it suitable for a beginner while leaving clear opportunities for future expansion.

20. Declaration

This report describes the current educational implementation of the University Payroll Management System. The student should review the implementation, understand the calculations and program flow, and personalize the final submission according to the university's academic requirements.